Unit Testing and Integration Testing

Software testing is used to find errors and ensure the software works correctly. Two important levels of testing are:

- Unit Testing
- Integration Testing

These are done in early stages of development to improve software quality.

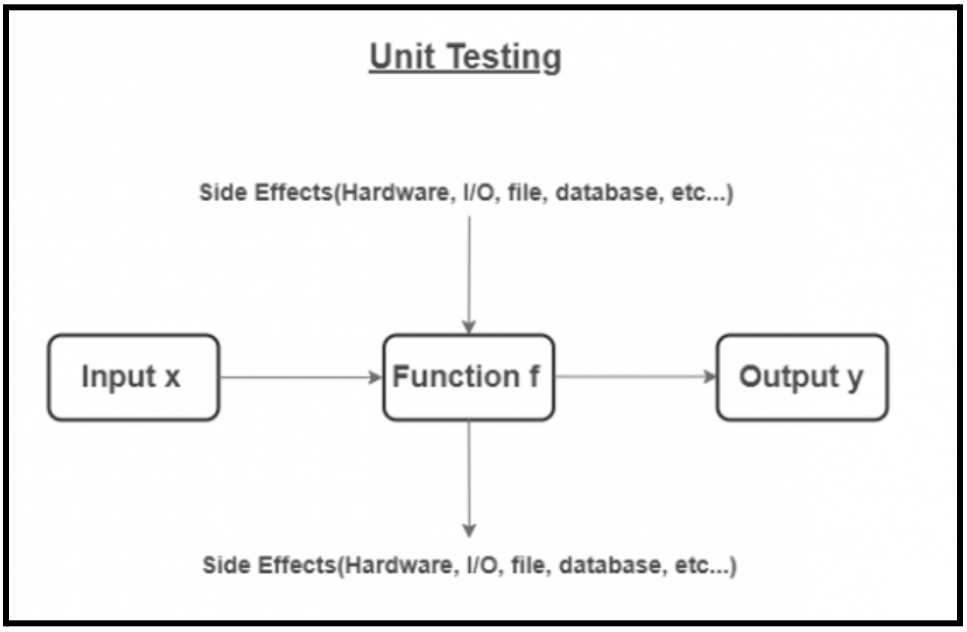
---

1. Unit Testing

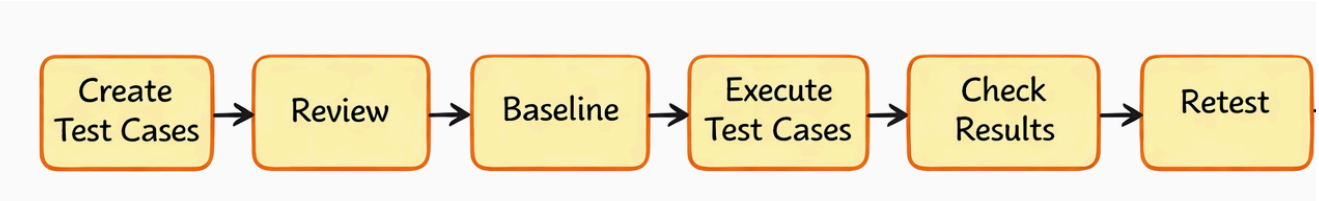
Unit testing is a type of testing where individual modules or components of software are tested separately. It focuses on small parts of the program.

Objectives of Unit Testing

- ❖ To test each function or method
- ❖ To verify correctness of code
- ❖ To detect bugs early
- ❖ To improve code quality
- ❖ To help developers understand the code



Workflow of Unit Testing



Types of Unit Testing Techniques

1. Black Box Testing

- Also called **behavioral testing**
- Focuses on input and output
- Test cases are derived from requirements
- Does not consider internal structure

Errors Found:

- Missing functions
- Interface errors
- Data structure errors
- Performance errors
- Initialization

---

2. White Box Testing

- Also called **structural testing**
- Tests internal structure of program
- Requires knowledge of code

Types:

a) Condition Testing

- Tests logical conditions in program
- Checks true/false conditions

b) Loop Testing

- Tests loops in program
- Types of loops:
  - Simple loop
  - Nested loop
  - Concatenated loop
  - Unstructured loop

Unit Testing Tools

- JUnit
- NUnit
- TestNG
- Emma
- PHPUnit

Advantages of Unit Testing

1) Helps developers learn code behavior
2) Ensures module works properly
3) Allows testing before full system completion
4) Early detection of errors
5) Improves code quality

Disadvantages of Unit Testing

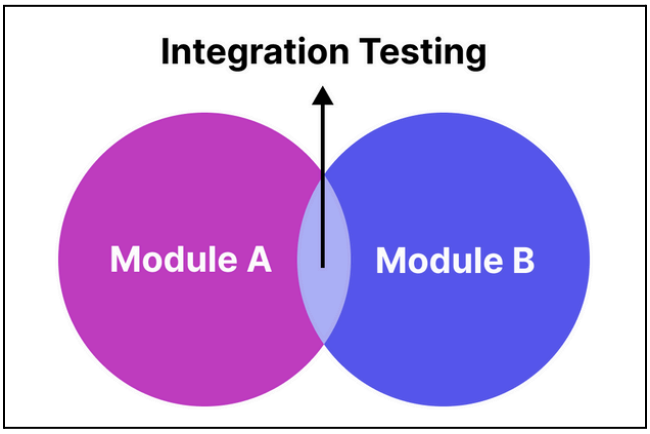
1. 1Time consuming
2. Cannot detect all errors
3. Errors may appear during integration

---

2. Integration Testing

1. Definition

Integration testing is the process of testing combined modules to verify their interaction. It focuses specifically on the interfaces between integrated units. This level of testing is designed to find errors in the way components work together.

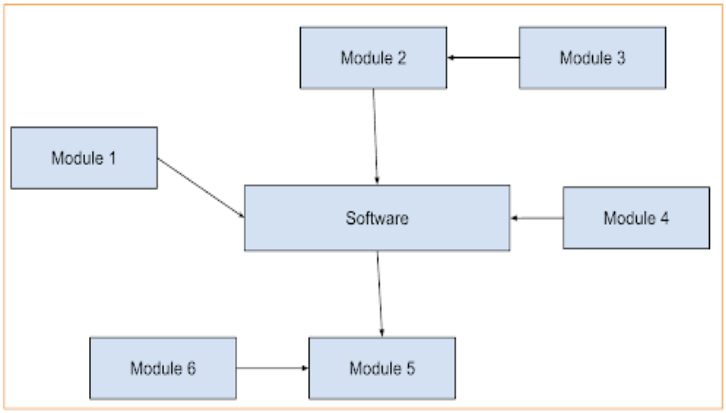


Purpose of Integration Testing

- To expose faults in interaction between modules
- To verify combined functionality
- To ensure proper data flow

Types of Integration Testing

1. Big Bang Integration Testing



- All modules are integrated at once
- Entire system tested as a **whole**

Advantages:

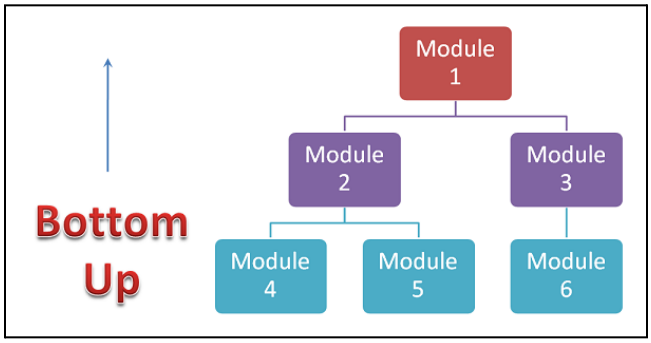
- Simple approach
- Suitable for small systems
- No planning require

Disadvantages:

- Difficult to locate errors
- Debugging is hard
- Errors may belong to any module

---

2. Bottom-Up Integration Testing



- Testing starts from **lower level modules**
- Modules are integrated step by step
- Uses **drivers**

Advantages:

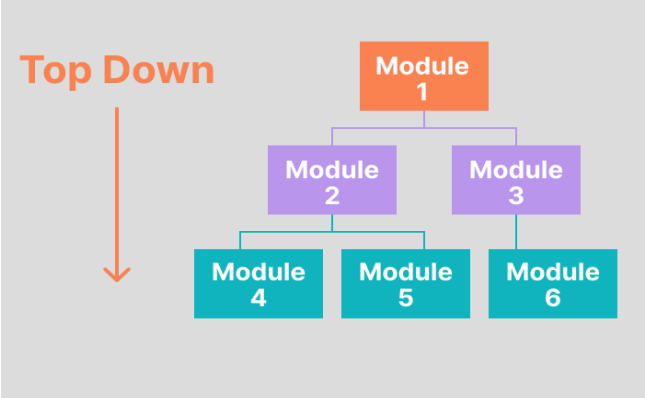
- No stubs required
- Easy to detect errors
- Suitable for bottom-up design

Disadvantages:

- Drivers must be developed
- High-level modules tested late

3. Top-Down Integration Testing

- Testing starts from **top modules**
- Uses **stubs**
- Lower modules added step by step



Advantages:

- Important modules tested early
- No drivers required
- Easy to find interface errors

Disadvantages:

- Requires many stubs
- Low-level modules tested later
- Difficult to design test cases

Regression Testing

Regression testing is the systematic process of verifying modified code and existing unchanged components to ensure that recent changes have not introduced new errors or regressions. This procedure is initiated when software modifications occur, including:

- Adding new features
- Fixing bugs
- Optimization

---

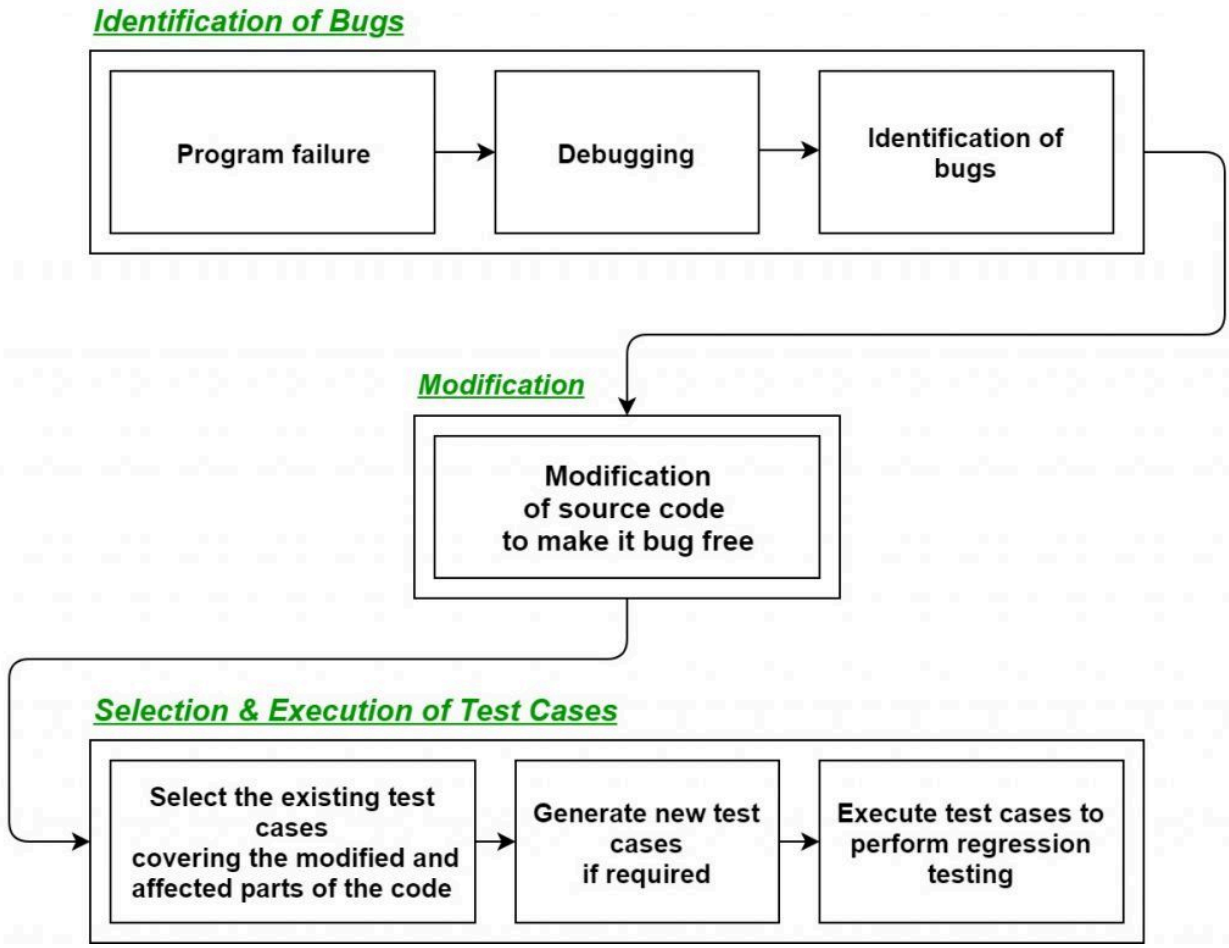
2. Purpose of Regression Testing

- To ensure modifications do not negatively impact existing functionality
- To verify that legacy test cases continue to execute correctly
- To identify new defects resulting from modifications

3. Process of Regression Testing

1) Modify the source code
2) Execute previously established test cases
3) Analyze results
4) Identify bugs
5) Perform debugging on the program

Identification of Targets, Select & Execute Test Cases



Prioritization of Test Cases

Based on Technical Requirements:

1) Essential test cases
2) Important test cases
3) Execute resource planning
4) Less important test cases
5) Redundant test cases

Regression Testing Tools

- Selenium
- Watir
- QTP
- RFT
- WinRunner
- SilkTest

Advantages of Regression Testing

1) Ensures that new modifications do not disrupt established features
2) Aids in maintaining overall software excellence
3) Detects structural defects post-modification

Disadvantages of Regression Testing

1) High consumption of time and resources
2) Necessity for automation tools in large systems
3) Lack of efficiency regarding minor adjustments

Compare White Box Testing and Black Box Testing

Introduction

Software testing is utilized to identify defects and verify that the application operates as intended. Two primary methodologies include:

1. White Box Testing
2. Black Box Testing

Both strategies are essential for enhancing software quality during the development lifecycle.

---

1. White Box Testing

White Box Testing, also known as structural testing, involves verifying the internal code, logic, and execution paths of a program.

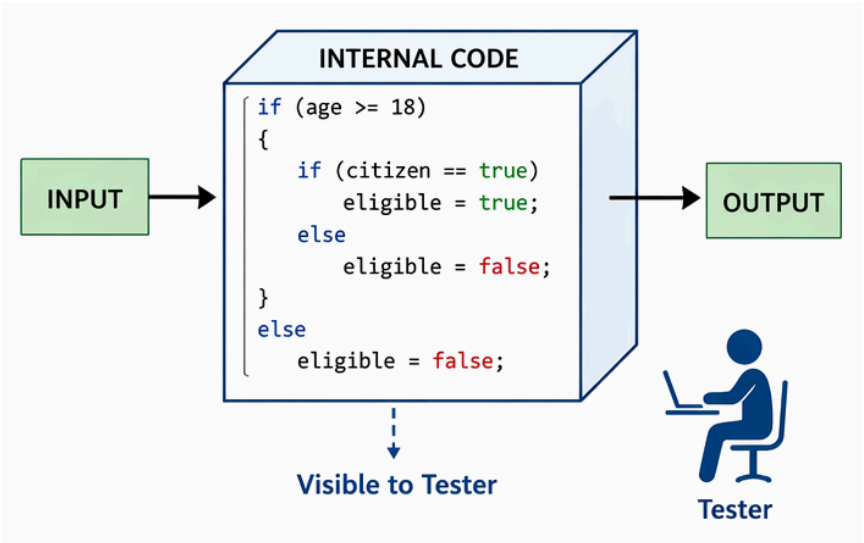
The tester possesses full knowledge of the source code and internal implementation.

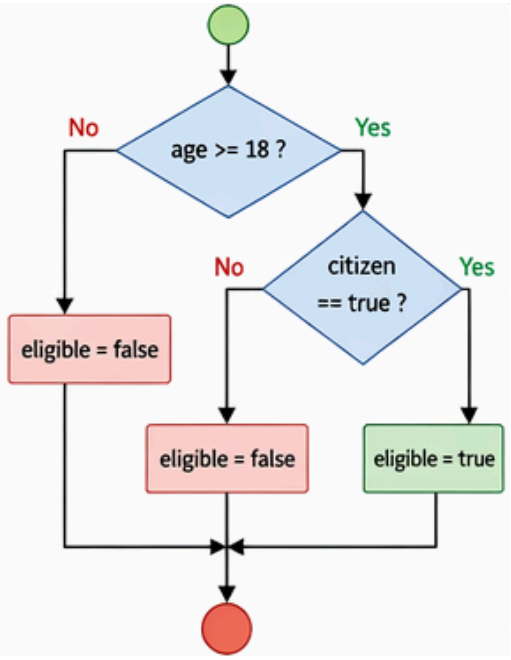
Focus Areas:

- Logical conditions
- Program loops
- Code statements
- Execution paths
- Internal logic structures

Example:

Analyzing whether the internal login validation code correctly processes authentication requests.





2. Black Box Testing

Black Box Testing, or behavioral testing, focuses on the software's functionality without regarding the internal code structure.

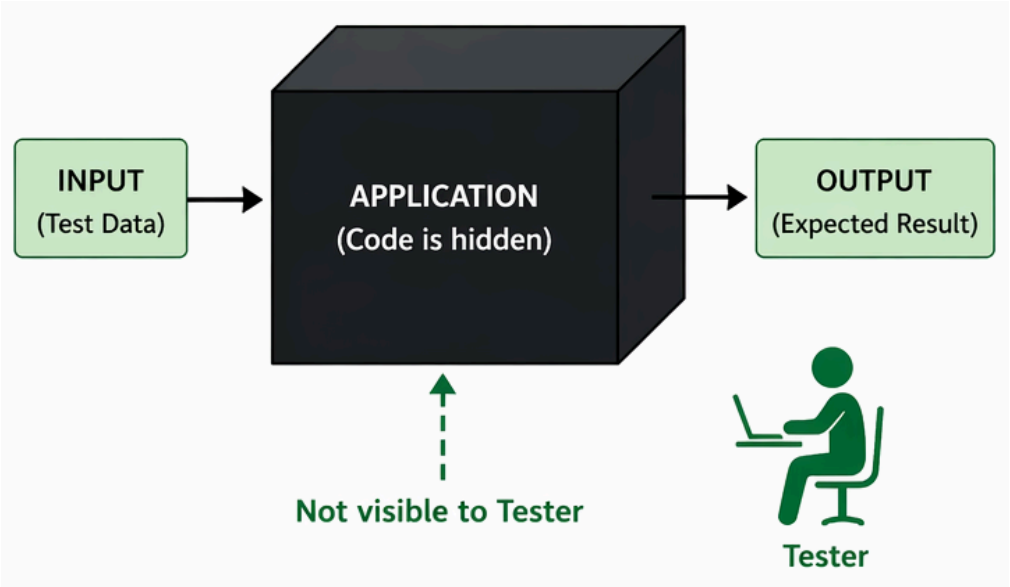
The tester examines only the inputs provided and the resulting outputs.

Focus Areas:

- Functional behavior
- User requirements compliance
- Validation of input values
- Verification of output results

Example:

Submitting a username and password to verify if the login interface grants or denies access correctly.



Test Case	Input	Expected Output
1	Valid Login	Success
2	Invalid Password	Failure
3	Empty Username	Failure
4	Empty Password	Failure
5	Special Characters	Failure

Comparison: White Box vs. Black Box Testing

White Box Testing	Black Box Testing
Evaluates internal code structure	Evaluates external functionality
Tester is cognizant of source code	Tester is unaware of code details
Focuses on execution paths and logic	Focuses on inputs and expected outputs
Known as Clear Box or Glass Box testing	Known as Functional testing
Typically executed by developers	Typically executed by QA testers
Necessitates programming expertise	No programming expertise required
Identifies errors in the source code	Identifies missing or incorrect features
Primarily utilized during unit testing	Utilized during system/integration testing
Resource-intensive for complex code	Facilitates user-centric validation
Verifies loops and logical conditions	Verifies functional requirements

---

White Box Testing Techniques

1. Statement Coverage
2. Branch Coverage
3. Path Coverage
4. Condition Testing
5. Loop Testing

Black Box Testing Techniques

1. Equivalence Partitioning
2. Boundary Value Analysis
3. Decision Table Testing
4. Error Guessing
5. State Transition Testing

Testing Type	Advantages	Disadvantages
White Box Testing	1. Finds hidden coding errors easily 2. Check all logic paths properly 3. Improves code quality 4. Helps remove extra or unused code.	1. Needs programming knowledge 2. It takes more time. 3. Difficult for large systems.
Black Box Testing	1. No coding knowledge needed. 2. Tests from user point of view. 3. Finds missing requirements. 4. Easy to perform.	1. Internal code is not checked. 2. Some internal paths may be missed. 3. Impossible to test all input cases.